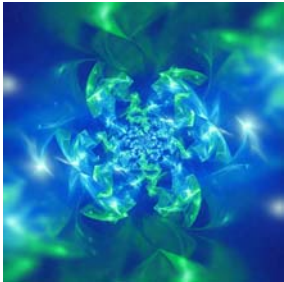


B 2.4.4 Recursion (HL Only)



B 2.4.4 Explain the fundamental concept of recursion and its applications in programming. (HL only)

Unlock the power of recursion! This section explores the fascinating world of recursive programming. Learn how functions can call themselves to solve problems, discover the advantages and limitations of this technique, and see how it's applied in real-world scenarios.

Learning objectives

- Students can define and explain the concept of recursion in programming.
- Students can identify the base and recursive cases in a recursive algorithm.
- Students will be able to analyse recursive algorithms' time and space complexity.
- Students will be able to evaluate the advantages and disadvantages of using recursion.
- Students can apply recursion to solve problems that can be broken down into smaller, similar subproblems.

IB ATL Skills

Thinking: Critical thinking, creative thinking, transfer

Research: Information literacy

Communication: Communication, collaboration

Self-Management: Organization, reflection

Lesson Plan (2 hours):

Part 1: Introduction to Recursion (30 minutes)

Engage (10 minutes)

Real-world analogy: Introduce recursion using a real-world analogy, such as a set of Russian nesting dolls or a mirror reflecting another mirror.

Discussion: Facilitate a brief discussion about self-reference and how it relates to recursion in programming.

Explore (10 minutes)

Define recursion: Explain recursion as a programming technique where a function calls itself to solve a problem.

Key components: Introduce the concepts of the base case (the condition that stops the recursion) and the recursive case (the step where the function calls itself with a smaller subproblem).

Visual Aid: Use diagrams or flowcharts to illustrate the execution flow in a recursive function.

Explain (10 minutes)

Simple example: Demonstrate a simple recursive function, such as calculating the factorial of a number. Provide code examples in both Java and Python.

Tracing: Work through the execution of the factorial function step-by-step, showing how it calls itself with smaller values until it reaches the base case.

Part 2: Advantages, Disadvantages, and Applications (30 minutes)

Explain (15 minutes)

Advantages of recursion: Discuss the advantages of recursion, such as code readability and elegance, especially for problems that can be naturally broken down into smaller, self-similar subproblems.

Disadvantages of recursion: Introduce the potential drawbacks of recursion, including stack overflow errors and potential performance overhead.

Elaborate (15 minutes)

Applications: Show examples of real-world applications of recursion, such as fractal image generation, traversing tree structures, and implementing sorting algorithms like quicksort.

Visualizations: Use visualizations to illustrate how recursion works in these applications (e.g., a fractal image being generated, a tree being traversed).

Part 3: Limitations and When to Use Recursion (30 minutes)

Explain (15 minutes)

Limitations of recursion: Discuss the limitations of recursion, particularly in terms of potential stack overflow errors and performance overhead compared to iterative solutions.

Memory usage: Explain how recursion uses stack memory to store function calls and how excessive recursion can lead to memory issues.

Elaborate (15 minutes)

When to use recursion: Discuss situations where recursion is most suitable, such as when the problem can be naturally broken down into smaller, self-similar subproblems and when code readability and elegance are prioritized.

Part 4: Activity/Worksheet and Q&A (30 minutes)

Evaluate (15 minutes)

Activity/Worksheet: Provide an activity or worksheet that allows students to explore recursion's advantages, disadvantages, and limitations. This could involve analysing code snippets, comparing recursive and iterative solutions, or identifying potential issues in recursive functions.

Conclusion (15 minutes)

Q&A: Facilitate a class discussion to review key concepts and answer student questions about recursion.

Key takeaways: Summarize the fundamental recursion principles, their advantages and disadvantages, and when they are most appropriate.

Lesson Resources

Presentations

Java

[Link to presentation](#)

Python

[Link to presentation](#)

eBook

The following eBook pages can be used by students to revise or review this topic.

[B 2.4.4 Java](#)

[B 2.4.4 Python](#)

Worksheets

The links below provide worksheets to accompany this section. An answer key is provided for both worksheets.

[B 2.4.4 Worksheet Java](#)

[B 2.4.4 Worksheet Python](#)

[B 2.4.4 Worksheet Java Answers](#)

[B 2.4.4 Worksheet Python Answers](#)

All materials on this website are for the exclusive use of teachers and students at subscribing schools for the period of their subscription. Any unauthorised copying or posting of materials on other websites is an infringement of our copyright and could result in your account being blocked and legal action being taken against you.

 **Report a problem**